CAHIER D'ÉTUDES ET DE RECHERCHES

PHP Objets et BDD



```
<!DOCTYPE html>
<html>
    <head>
        <title>Cours PHP & MySQL</title>
        <meta charset="utf-8">
        <meta name="viewport"
          content="width=device-width, initial-scale=1, user-scalable=no">
        <link rel="stylesheet" href="cours.css">
    </head>

    <body>
        <h1>Titre principal</h1>
        <?php
            require 'classes/utilisateur.class.php';

            new Utilisateur();
        ?>
        <p>Un paragraphe</p>
    </body>
</html>
```

LOUBOVE Dan Célestin

12/10/2026

X2030

UCAC | icam

Table des matières

PHP Objets et BDD....1
Table des matières....2
I. Prendre connaissance de la situation....5
1. Mots-clés....5
2. Contexte....5
II. Analyse des besoins....5
1. Besoins....5
2. Contraintes....5
3. Problématique....5
III. Généralisation....6
IV. Pistes de solution....6
V. Plan d'action....6
VI. Réalisation du plan d'action....7
1. Définir les mots-clés....7
PHP....7
POO - Programmation Orienté Objet....7
Classe et objet....8
Héritage....8
Encapsulation....9
Polymorphisme....10
MySQL....11
Base de données relationnelles....11
PDO....11
2. Étude de la POO en PHP....12
Les modificateurs d'accès....12
Le constructeur....12
Les méthodes statiques....13
Les interfaces et les classes abstraites....14
L'organisation des classes dans un projet....14
3. Étudier les principes généraux de la POO....14
Les cinq principes à maîtriser....15
Comment les objets collaborent-ils ?....15
Client HTTP....15
ArchiveController....15
ArchiveRepository....15
MySQL....15
4. Étude des bases de données relationnelles....16
Concevoir le modèle de données....16

Définir les relations....16
UTILISATEURS....16
ARCHIVES....16
CATÉGORIES....16
Comprendre les clés....16
Comprendre les opérations SQL....17
5. Adaptation de l'API à la structure orientée objet....17
Créer l'arborescence du projet....18
Créer la classe `Archive` 18
Créer la classe `Database` 19
Créer le dépôt `ArchiveRepository` 20
Pourquoi utiliser `prepare()` et `execute()` ?....22
Créer le contrôleur....22
Créer le point d'entrée de l'API....26
6. Créer la base de donnée....28
Écrire le script SQL....28
Exécuter le script avec phpMyAdmin....30
Vérifier les données....30
7. Faire communiquer MySQL avec PHP orienté objet grâce à PDO....31
Tester la connexion à MySQL....31
Tester l'API avec les différentes méthodes HTTP....31
GET....32
Récupérer toutes les archives....32
Récupérer une archive....32
POST....32
Créer une archive....32
PUT....32
Modifier une archive....32
DELETE....33
Supprimer une archive....33
Pour créer une archive depuis un terminal :....33
Pour consulter les archives :....33
Pour modifier une archive :....33
Pour supprimer une archive :....33
8. Validation des hypothèses....34
9. Bilan du travail....34
Objectifs atteints....34
Bilan individuel....34
Bilan collectif....34

10. Synthèse du travail et résultats.....35
Compétences acquises..... 35
Résultats obtenus..... 35
Perspectives..... 42
8. Références bibliographiques..... 43
Références fournies par le prosit.....43
Ressources Théoriques :..... 43
1. Documentation PHP sur la Programmation Orientée Objet (POO) :.... 43
• PHP.net - Programmation Orientée Objet (POO).....43
• PHP.net - Classes et Objets.....43
2. PDO (PHP Data Objects) pour la connexion à MySQL :.....43
• PHP.net - Introduction à PDO..... 43
• PHP.net - Utilisation des requêtes préparées avec PDO.....43
3. Tutoriels PHP sur les concepts de POO :.....43
• W3Schools - PHP OOP Tutorial.....43
• TutorialsPoint - PHP OOP Tutorial.....43
4. Tutoriels sur l'intégration PHP/MySQL avec PDO :..... 43
• DigitalOcean - Guide pour connecter PHP à MySQL avec PDO.....43
• W3Schools - PHP MySQL Database Tutorial.....43
5. Guides de sécurité des applications Web :.....43
• OWASP - Guide de prévention des injections SQL..... 43
• OWASP - Prévention des attaques CSRF.....43
• OWASP - Guide de prévention des attaques XSS..... 43
Ressources Pratiques :..... 43
1. Exemples de code et projets similaires :..... 43
• GitHub - Exemples de projets PHP/MySQL utilisant la POO.....43
• CodeCanyon - Exemples d'applications PHP sécurisées..... 43
2. Validation et assainissement des données en PHP :.....43
• PHP.net - Validation de données.....43
• W3Schools - PHP Security.....43
3. Génération et validation de tokens CSRF :.....43
• OWASP - Token CSRF.....44
• CSRF Token Tutorial en PHP..... 44
Outils utilisés..... 44

I. Prendre connaissance de la situation

1. Mots-clés

- PDO
- POO
- MySQL
- Les classes
- Objets
- Heritage
- Encapsulation
- Polymorphisme
- Base de données relationnelle

2. Contexte

Les quatre étudiants de codeland, ayant déjà configurer leur environnement PHP et crée leurs premières API pour leur plateforme d'archivages concertent pour restructurer leur plateforme d'archivage en poo et y intégrer une base de données MySQL via PDO afin de rendre leur code plus modulaire et mieux gérer leurs données.

II. Analyse des besoins

1. Besoins

- Connaissance sur la PDO en PHP
- Connaissance sur la gestion des bases de données
- Connaissances sur l'utilisation de la POO

2. Contraintes

- L'utilisation de la PDO
- L'intégration d'une base de données relationnelle
- L'utilisation de la POO

3. Problématique

Comment les quatre étudiants de Codeland doivent restructurer leur plateforme d'archivage en POO et y intégrer une base de données MySQL via PDO, afin de rendre le code plus modulaire et de mieux gérer les données ?

III. Généralisation

PHP ORIENTE OBJET ET GESTION DES BD

IV. Pistes de solution

Devons-nous :

- Utiliser postages pour la BD ?
- Utiliser une autre méthode en plus de PDO pour la sécurité de la BD ?
- Mettre en place des fonctions spécifiques pour gérer la BD ?

V. Plan d'action

I. Définir les mots clés
II. Etudier les POO en PHP
III. Etudier les POO
IV. Etudier les BD relationnelles
V. Adapter l'API a la structure POO
VI. Créer la base de données
VII. Faire communiquer la BD relationnelle avec PHP oriente objet via POO

VI. Réalisation du plan d'action

1. Définir les mots-clés

PHP

PHP est un langage de programmation principalement utilisé pour développer des applications web côté serveur. Il permet de recevoir des requêtes HTTP, de traiter les données, de communiquer avec une base de données et de retourner des réponses, notamment au format JSON pour une API.

Exemple :

```
<?php
$nom = "Codeland";

echo "Bienvenue sur " . $nom;
```

POO - Programmation Orienté Objet

La POO est un paradigme de programmation qui organise le code autour d'objets. Elle permet de regrouper les données et les traitements associés, afin de rendre une application plus modulaire, réutilisable et facile à maintenir.

Les quatre notions essentielles sont :

- Encapsulation : regrouper les données et les méthodes dans une classe, tout en contrôlant leur accès.
- Héritage : permettre à une classe de reprendre les propriétés et méthodes d'une autre classe.
- Polymorphisme : permettre à des objets de classes différentes de répondre à une même méthode selon leur propre comportement.
- Abstraction : exposer les fonctionnalités essentielles tout en masquant les détails d'implémentation.

Classe et objet

Build déclenché automatiquement (commit, merge, planning) sur un serveur dédié, avec les mêmes étapes à chaque fois

Une classe est un modèle qui définit les propriétés et les méthodes d'un type d'objet. Un objet est une instance concrète de cette classe.

```
<?php
class Archive
{
   public string $titre;

   public function __construct(string $titre)
   {
      $this->titre = $titre;
   }

   public function afficherTitre(): string
   {
      return $this->titre;
   }
}

$archive = new Archive("Cours de PHP");

echo $archive->afficherTitre();
```

Ici, `Archive` est la classe, `$archive` est un objet, `titre` est une propriété et `afficherTitre()` est une méthode.

Héritage

L'héritage permet de créer une classe spécialisée à partir d'une classe existante.

```
<?php
class Document
{
   public function afficherType(): string
   {
      return "Document";
```

```
    }
}

class Rapport extends Document
{
    public function afficherType(): string
    {
        return "Rapport";
    }
}

$rapport = new Rapport();
echo $rapport->afficherType();
```

Résultat : `Rapport`.

La classe `Rapport` hérite de `Document` et redéfinit sa méthode.

Encapsulation

L'encapsulation protège les données d'un objet en contrôlant les accès à ses propriétés.

```
<?php
class Utilisateur
{
    private string $nom;

    public function __construct(string $nom)
    {
        $this->nom = $nom;
    }

    public function getNom(): string
    {
        return $this->nom;
    }
}

$user = new Utilisateur("Paul");
echo $user->getNom();
```

La propriété `$nom` est privée : elle n'est pas accessible directement depuis l'extérieur de la classe.

Polymorphisme

Le polymorphisme permet d'utiliser une même interface de programmation avec plusieurs comportements.

```
<?php
interface Exportable
{
   public function exporter(): string;
}

class ExportPDF implements Exportable
{
   public function exporter(): string
   {
      return "Exportation au format PDF";
   }
}

class ExportTexte implements Exportable
{
   public function exporter(): string
   {
      return "Exportation au format texte";
   }
}

function lancerExport(Exportable $document): void
{
   echo $document->exporter();
}

lancerExport(new ExportPDF());
lancerExport(new ExportTexte());
```

La fonction `lancerExport()` accepte plusieurs types d'objets, à condition qu'ils respectent le contrat `Exportable`.

MySQL

MySQL est un système de gestion de base de données relationnelle (SGBDR). Il stocke les informations dans des tables composées de lignes et de colonnes.

Exemple de table `archives` :

id	titre	type
1	COURS PHP	PDF
2	Rapport projet	DOCX

Chaque ligne représente une archive et chaque colonne une caractéristique de cette archive.

Base de données relationnelles

Une base de données relationnelle organise les données dans des tables liées entre elles grâce à des clés.

Par exemple :

- `utilisateurs` : les personnes qui déposent les archives ;
- `archives` : les documents archivés ;
- `categories` : les catégories des documents.

Une clé étrangère permet d'associer une archive à son utilisateur ou à sa catégorie.

PDO

PDO signifie PHP Data Objects. C'est une extension de PHP qui fournit une interface pour communiquer avec une base de données, notamment MySQL.

PDO permet de :

- se connecter à une base de données ;
- exécuter des requêtes SQL ;
- récupérer les résultats ;
- utiliser des requêtes préparées pour limiter les risques d'injection SQL ;
- gérer les transactions.

Exemple :

```
<?php
$pdo = new PDO(
   "mysql:host=localhost;dbname=codeland;charset=utf8mb4",
   "root",
   "",
   [
      PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
      PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
   ]
);
```

Cette connexion suppose que MySQL fonctionne localement et que les identifiants correspondent à votre configuration.

2. Étude de la POO en PHP

Les modificateurs d'accès

Modificateur	Accessibilité
public	Accessible depuis l'extérieur et l'intérieur de la classe
protected	Accessible dans la classe et ses classes dérivées
private	Accessible uniquement dans la classe qui le déclare

Le constructeur

La méthode `__construct()` est appelée automatiquement lorsqu'un objet est créé.

```
<?php

class Archive

{
```

```
    public function __construct(
        public string $titre,
        public string $type
    ) {}
}

$archive = new Archive("Cours PHP", "PDF");

echo $archive->titre;
```

La syntaxe utilisée ici est une fonctionnalité moderne de PHP qui déclare automatiquement les propriétés à partir des paramètres du constructeur.

Les méthodes statiques

Une méthode statique appartient à la classe et peut être appelée sans créer d'objet.

```
<?php
class Validateur
{
    public static function titreValide(string $titre): bool
    {
        return trim($titre) !== "";
    }
}

var_dump(Validateur::titreValide("Cours PHP"));
```

Les interfaces et les classes abstraites

- Une interface définit les méthodes qu'une classe doit implémenter.
- Une classe abstraite peut définir des méthodes communes et imposer certaines méthodes aux classes dérivées.

Ces mécanismes facilitent la séparation des responsabilités dans une application.

L'organisation des classes dans un projet

Pour Codeland, on peut organiser les fichiers ainsi :

```
text
codeland/
├── config/
│   └── Database.php
├── models/
│   └── Archive.php
├── repositories/
│   └── ArchiveRepository.php
├── controllers/
│   └── ArchiveController.php
├── routes/
│   └── archives.php
└── public/
    └── index.php
```

Cette structure distingue la connexion à la base de données, la représentation des archives, les opérations SQL, la logique de traitement et les points d'entrée de l'API.

3. Étudier les principes généraux de la POO

Cette étape complète la précédente : il faut comprendre comment concevoir les classes et faire collaborer les objets pour éviter d'écrire toute l'application dans un seul fichier.

Les cinq principes à maîtriser

- Responsabilité unique : une classe doit avoir un rôle principal. Par exemple, `Database` gère la connexion, tandis que `ArchiveRepository` effectue les opérations sur les archives.

- Encapsulation : les données internes sont protégées et manipulées au moyen de méthodes contrôlées.
- Réutilisabilité : une classe peut être utilisée dans plusieurs parties du projet.
- Héritage et polymorphisme : ils permettent de partager ou de spécialiser des comportements lorsque cela est utile.
- Faible couplage : modifier une classe doit avoir le moins d'effets possible sur les autres.

Comment les objets collaborent-ils ?

Dans notre API d'archivage, le fonctionnement sera le suivant :

Client HTTP

Envoie une requête GET, POST, PUT ou DELETE

ArchiveController

Interprète la requête et prépare la réponse JSON

ArchiveRepository

Exécute les opérations SQL avec PDO

MySQL

Stocke et retourne les données

L'intérêt est de pouvoir modifier la manière de stocker les archives sans réécrire toute la gestion des requêtes HTTP.

4. Étude des bases de données relationnelles

Concevoir le modèle de données

Nous allons utiliser trois tables pour la plateforme Codeland :

- `utilisateurs` : les personnes qui déposent des archives ;
- `categories` : les catégories de documents ;

- `archives` : les documents enregistrés.

Définir les relations

UTILISATEURS

- `id` — clé primaire
- `nom`, `email`

Un utilisateur possède plusieurs archives

ARCHIVES

- `id` — clé primaire
- `titre`, `description`, `fichier_url`
- `utilisateur_id` — clé étrangère
- `categorie_id` — clé étrangère

Une catégorie regroupe plusieurs archives

CATÉGORIES

- `id` — clé primaire
- `nom` — nom unique de la catégorie

Comprendre les clés

- Clé primaire (`PRIMARY KEY`) : identifie de manière unique chaque ligne.
- Clé étrangère (`FOREIGN KEY`) : garantit qu'une référence pointe vers un enregistrement existant.
- Contrainte `UNIQUE` : empêche les doublons sur une colonne, par exemple l'adresse e-mail.

- Contrainte `NOT NULL` : oblige à fournir une valeur.
- Index : accélère certaines recherches.

Comprendre les opérations SQL

Opération	SQL	Utilité
Créer	INSERT	Ajouter une archive
Lire	SELECT	Consulter les archives
Modifier	UPDATE	Changer le titre ou la description
Supprimer	DELETE	Supprimer une archive

Il s'agit des quatre opérations CRUD : Create, Read, Update, Delete.

5. Adaptation de l'API à la structure orientée objet

Nous allons maintenant commencer l'implémentation. L'objectif est d'obtenir une API PHP native, sans framework complexe, utilisant PDO et MySQL.

Créer l'arborescence du projet

Dans le dossier du projet,nous créeons la structure suivante :

text
codeland/
├── config/
│ └── Database.php
├── models/

```
│   └── Archive.php
├── repositories/
│   └── ArchiveRepository.php
├── controllers/
│   └── ArchiveController.php
├── public/
│   └── index.php
└── database/
    └── codeland.sql
```

Chaque fichier aura une responsabilité précise.

Créer la classe `Archive`

Fichier `models/Archive.php` :

```php
<?php

class Archive
{
    public function __construct(
        public string $titre,
        public ?string $description = null,
        public ?string $fichier_url = null,
        public ?int $categorie_id = null,
        public ?int $utilisateur_id = null,
        public ?int $id = null
    ) {}
}
```

Cette classe représente une archive dans l'application. Elle ne se connecte pas à MySQL et n'exécute aucune requête SQL.

Créer la classe `Database`

Fichier `config/Database.php` :

```php
<?php

class Database
```

```
{
    private PDO $pdo;

    public function __construct()
    {
        $host = "localhost";
        $dbname = "codeland";
        $username = "root";
        $password = "";

        dsn = "mysql: host =host;dbname=$dbname;charset=utf8mb4";

        $this->pdo = new PDO(
            $dsn,
            $username,
            $password,
            [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false
            ]
        );
    }

    public function getConnection(): PDO
    {
        return $this->pdo;
    }
}
```

Explication :

- `private PDO $pdo` stocke la connexion dans un attribut privé.
- `new PDO(...)` établit la connexion à MySQL.
- `PDO::ERRMODE_EXCEPTION` permet de signaler les erreurs SQL par des exceptions.
- `getConnection()` fournit la connexion aux classes qui en ont besoin.

Créer le dépôt `ArchiveRepository`

Le dépôt est chargé d'effectuer les opérations sur la table `archives`.

Fichier `repositories/ArchiveRepository.php` :

```
<?php

require_once __DIR__ . '/../models/Archive.php';

class ArchiveRepository
{
    public function __construct(private PDO $pdo) {}

    public function findAll(): array
    {
        $sql = "SELECT * FROM archives ORDER BY id DESC";

        return this -> pdo -> query(sql)->fetchAll();
    }

    public function findById(int $id): array|false
    {
        $stmt = $this->pdo->prepare(
            "SELECT * FROM archives WHERE id = :id"
        );

        $stmt->execute(['id' => $id]);

        return $stmt->fetch();
    }

    public function create(Archive $archive): int
    {
        $sql = "INSERT INTO archives
            (titre, description, fichier_url, categorie_id, utilisateur_id)
            VALUES
            (:titre, :description, :fichier_url, :categorie_id, :utilisateur_id)";

        $stmt = this -> pdo -> prepare(sql);

        $stmt->execute([
```

```
            'titre' => $archive->titre,
            'description' => $archive->description,
            'fichier_url' => $archive->fichier_url,
            'categorie_id' => $archive->categorie_id,
            'utilisateur_id' => $archive->utilisateur_id
        ]);

        return (int) $this->pdo->lastInsertId();
    }

    public function update(int $id, array $data): bool
    {
        $sql = "UPDATE archives
                SET titre = :titre,
                    description = :description,
                    fichier_url = :fichier_url,
                    categorie_id = :categorie_id
                WHERE id = :id";

        $stmt = this -> pdo -> prepare(sql);

        $stmt->execute([
            'titre' => $data['titre'],
            'description' => $data['description'] ?? null,
            'fichier_url' => $data['fichier_url'] ?? null,
            'categorie_id' => $data['categorie_id'] ?? null,
            'id' => $id
        ]);

        return $stmt->rowCount() > 0;
    }

    public function delete(int $id): bool
    {
        $stmt = $this->pdo->prepare(
            "DELETE FROM archives WHERE id = :id"
        );

        $stmt->execute(['id' => $id]);

        return $stmt->rowCount() > 0;
    }
```

```
}
```

Pourquoi utiliser `prepare()` et `execute()` ?

Au lieu de concaténer directement une valeur fournie par un utilisateur dans le SQL, on utilise un paramètre comme `:id`. Cela limite les risques d'injection SQL.

Attention : `rowCount()` peut retourner `0` si une modification ne change aucune valeur, même si l'enregistrement existe. Pour une API plus précise, on peut vérifier l'existence de l'enregistrement séparément.

Créer le contrôleur

Fichier `controllers/ArchiveController.php` :

```
<?php

require_once __DIR__ . '/../repositories/ArchiveRepository.php';

class ArchiveController
{
    public function __construct(
        private ArchiveRepository $repository
    ) {}

    private function json(array $data, int $status = 200): void
    {
        http_response_code($status);
        header('Content-Type: application/json; charset=utf-8');

        echo json_encode(
            $data,
            JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE
        );
```

```
}

public function index(): void
{
    this −> json(this->repository->findAll());
}

public function show(int $id): void
{
    $archive = this −> repository −> findById(id);

    if ($archive === false) {
        $this->json(['erreur' => 'Archive introuvable'], 404);
        return;
    }

    this −> json(archive);
}

public function store(): void
{
    $data = json_decode(
        file_get_contents('php://input'),
        true
    );

    if (
        !is_array($data) ||
        !isset($data['titre']) ||
        !is_string($data['titre']) ||
```

```
        trim($data['titre']) === ''
    ) {
        $this->json(['erreur' => 'Titre obligatoire'], 400);
        return;
    }

    $archive = new Archive(
        titre: trim($data['titre']),
        description: $data['description'] ?? null,
        fichier_url: $data['fichier_url'] ?? null,
        categorie_id: isset($data['categorie_id'])
            ? (int) $data['categorie_id']
            : null,
        utilisateur_id: isset($data['utilisateur_id'])
            ? (int) $data['utilisateur_id']
            : null
    );

    $id = this -> repository -> create(archive);

    $this->json([
        'message' => 'Archive créée',
        'id' => $id
    ], 201);
}

public function update(int $id): void
{
    $data = json_decode(
        file_get_contents('php://input'),
```

```
        true
    );

    if (
        !is_array($data) ||
        !isset($data['titre']) ||
        !is_string($data['titre']) ||
        trim($data['titre']) === ''
    ) {
        $this->json(['erreur' => 'Titre obligatoire'], 400);
        return;
    }

    if ($this -> repository -> findById(id) === false) {
        $this->json(['erreur' => 'Archive introuvable'], 404);
        return;
    }

    $this -> repository -> update(id, [
        'titre' => trim($data['titre']),
        'description' => $data['description'] ?? null,
        'fichier_url' => $data['fichier_url'] ?? null,
        'categorie_id' => isset($data['categorie_id'])
            ? (int) $data['categorie_id']
            : null
    ]);

    $this->json(['message' => 'Archive mise à jour']);
}
```

```
    public function destroy(int $id): void
    {
        if (!this -> repository -> delete(id)) {
            $this->json(['erreur' => 'Archive introuvable'], 404);
            return;
        }

        $this->json(['message' => 'Archive supprimée']);
    }
}
```

Le contrôleur fait le lien entre les requêtes HTTP et le dépôt. Dans une application de production, il faudra également valider les types, les longueurs des champs, les URL, les identifiants de catégorie et les droits de l'utilisateur.

Créer le point d'entrée de l'API

Fichier `public/index.php` :

```
<?php
require_once __DIR__ . '/../config/Database.php';
require_once __DIR__ . '/../controllers/ArchiveController.php';

try {
    $database = new Database();
    $repository = new ArchiveRepository(
        $database->getConnection()
    );
    controller = newArchiveController(repository);

    $method = $_SERVER['REQUEST_METHOD'];
    id = isset(_GET['id']) ? filter_var(
        $_GET['id'],
        FILTER_VALIDATE_INT,
        ['options' => ['min_range' => 1]]
    ) : null;
```

```
if (isset($_GET['id']) && $id === false) {
    http_response_code(400);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode(['erreur' => 'ID invalide']);
    exit;
}

switch ($method) {
    case 'GET':
        if ($id !== null) {
            controller -> show(id);
        } else {
            $controller->index();
        }
        break;

    case 'POST':
        $controller->store();
        break;

    case 'PUT':
        if ($id === null) {
            http_response_code(400);
            echo json_encode(['erreur' => 'ID obligatoire']);
            break;
        }

        controller -> update(id);
        break;

    case 'DELETE':
        if ($id === null) {
            http_response_code(400);
            echo json_encode(['erreur' => 'ID obligatoire']);
            break;
        }

        controller -> destroy(id);
        break;

    default:
        header('Allow: GET, POST, PUT, DELETE');
```

```
        http_response_code(405);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode(['erreur' => 'Méthode non autorisée']);
    }
} catch (Throwable $e) {
    error_log((string) $e);

    http_response_code(500);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode(['erreur' => 'Erreur interne du serveur']);
}
```

Le point d'entrée proposé utilise des paramètres de requête pour identifier une archive. Cela permet de tester simplement le fonctionnement sans configurer immédiatement un système de routage REST.

6. Créer la base de donnée

Nous allons maintenant créer les tables nécessaires à l'application.

Écrire le script SQL

Fichier `database/codeland.sql` :

```
CREATE DATABASE IF NOT EXISTS codeland
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE codeland;

CREATE TABLE utilisateurs (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL,
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE categories (
```

```
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE archives (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,
    description TEXT NULL,
    fichier_url VARCHAR(500) NULL,
    utilisateur_id INT UNSIGNED NULL,
    categorie_id INT UNSIGNED NULL,
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_archive_utilisateur
        FOREIGN KEY (utilisateur_id)
        REFERENCES utilisateurs(id)
        ON DELETE SET NULL,

    CONSTRAINT fk_archive_categorie
        FOREIGN KEY (categorie_id)
        REFERENCES categories(id)
        ON DELETE SET NULL
) ENGINE=InnoDB;

INSERT INTO categories (nom) VALUES
    ('Informatique'),
    ('Mathématiques'),
    ('Physique'),
    ('Rapports de projet');

INSERT INTO utilisateurs (nom, email, mot_de_passe) VALUES
    ('Étudiant Test', 'etudiant@example.com', 'A_REMPLACER_PAR_UN_HASH');

INSERT INTO archives (
    titre,
    description,
    fichier_url,
    utilisateur_id,
    categorie_id
) VALUES (
    'Introduction au PHP',
    'Document de démonstration',
```

```
    'uploads/introduction-php.pdf',
    1,
    1
);
```

Remarque importante : le compte d'exemple utilise une valeur provisoire pour le mot de passe. Elle ne doit pas servir à authentifier un utilisateur. En production, les mots de passe doivent être enregistrés avec `password_hash()` et vérifiés avec `password_verify()`.

Exécuter le script avec phpMyAdmin

- Ouvrez XAMPP et démarrez Apache et MySQL.
- Accédez à `http://localhost/phpmyadmin`.
- Ouvrez l'onglet SQL.
- Copiez le contenu du fichier `codeland.sql` et exécutez-le.
- Vérifiez que les tables `utilisateurs`, `categories` et `archives` ont bien été créées.

Vérifier les données

Exécutez cette requête :

```
SELECT
    a.id,
    a.titre,
    c.nom AS categorie,
    u.nom AS auteur
FROM archives a
LEFT JOIN categories c ON a.categorie_id = c.id
LEFT JOIN utilisateurs u ON a.utilisateur_id = u.id;
```

7. Faire communiquer MySQL avec PHP orienté objet grâce à PDO

Cette étape consiste à vérifier que toutes les parties développées fonctionnent ensemble.

Tester la connexion à MySQL

Créez temporairement un fichier `test-connexion.php` à la racine du projet :

```
<?php
require_once __DIR__ . '/config/Database.php';

try {
    $database = new Database();
    $pdo = $database->getConnection();

    echo "Connexion à MySQL réussie !";
} catch (PDOException $e) {
    echo "Échec de la connexion.";
}
```

Lancez Apache et ouvrez : `http://localhost/codeland/test-connexion.php`
Adaptez le chemin à l'emplacement réel de votre projet dans `htdocs`. Une fois le test effectué, supprimez ce fichier ou retirez-le de l'accès public.

Tester l'API avec les différentes méthodes HTTP

Pour les exemples ci-dessous, supposons que le projet se trouve dans `C:\xampp\htdocs\codeland`.

GET

Récupérer toutes les archives

`http://localhost/codeland/public/index.php`

Réponse attendue : un tableau JSON contenant les archives.

Récupérer une archive

`http://localhost/codeland/public/index.php?id=1`

Réponse attendue : les informations de l'archive numéro 1.

POST

Créer une archive

URL :
http://localhost/codeland/public/index.php

Envoyez un corps JSON :

```
{
  "titre": "Cours de Java",
  "description": "Introduction à la POO",
  "categorie_id": 1,
  "utilisateur_id": 1
}
```

PUT

Modifier une archive

`http://localhost/codeland/public/index.php?id=1`

```
{
  "titre": "Cours de PHP avancé",
  "description": "Version mise à jour",
  "categorie_id": 1
}
```

DELETE

Supprimer une archive

`http://localhost/codeland/public/index.php?id=1`

Envoyez une requête DELETE. Attention : cette opération supprime réellement l'enregistrement.

Vous pouvez effectuer ces tests dans Postman ou avec `curl`. Pour les requêtes POST et PUT, définissez l'en-tête `Content-Type: application/json`.

Exemple de test avec cURL

Pour créer une archive depuis un terminal :

```
curl -X POST \
  http://localhost/codeland/public/index.php \
  -H "Content-Type: application/json" \
      -d "{\"titre\":\"Cours de Java\",\"description\":\"Introduction à la POO\",\"categorie_id\":1,\"utilisateur_id\":1}"
```

Pour consulter les archives :

```
curl http://localhost/codeland/public/index.php
```

Pour modifier une archive :

```
curl -X PUT \
  "http://localhost/codeland/public/index.php?id=1" \
  -H "Content-Type: application/json" \
      -d "{\"titre\":\"Cours de PHP avancé\",\"description\":\"Version mise à jour\",\"categorie_id\":1}"
```

Pour supprimer une archive :

```
curl -X DELETE \
  "http://localhost/codeland/public/index.php?id=1"
```

8. Validation des hypothèses

Les pistes 1 et 3 ont été partiellement validées.

9. Bilan du travail

Objectifs atteints

Ce prosit visait à se familiariser avec la notion d'automatisation et de déploiement logiciel. Ce travail m'a permis de comprendre ces objectifs par le biais du workshop et de la réalisation de la solution qui ont permis la mise en pratique des connaissances acquises.

Bilan individuel

La réalisation de cette nouvelle version m'a permis de mieux comprendre les phases du de l'automatisation et le développement logiciel. J'ai appris à Définir et utiliser des classes et des objets en PHP, appliquer l'encapsulation, l'héritage et le polymorphisme, Conception des tables relationnelles et leurs clés, etc.

Bilan collectif

- Les discussions collectives nous ont permis de mieux comprendre certains concepts et de corriger nos erreurs de logique
- Le partage d'idées a enrichi notre compréhension mutuelle
- Les débats sur les différentes approches algorithmiques ont développé notre esprit critique
- L'entraide a permis de surmonter les difficultés individuelles
- Communication efficace entre les membres
- Respect du planning établi
- Partage équitable des tâches
- Solidarité dans la résolution des problèmes

10. Synthèse du travail et résultats

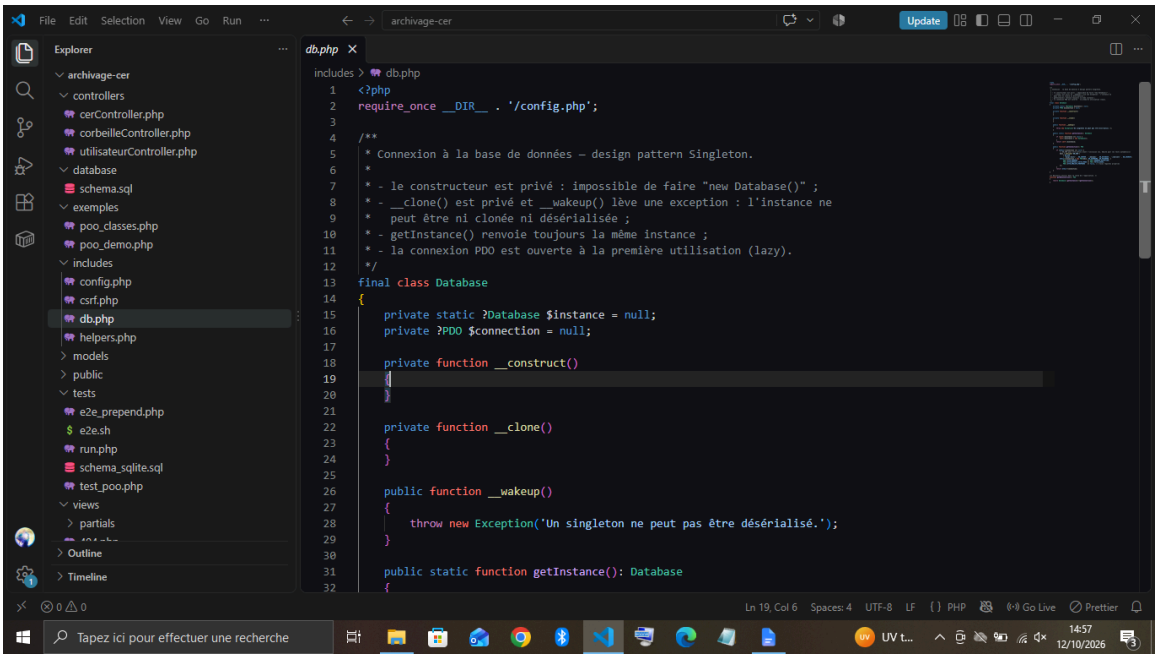
Compétences acquises

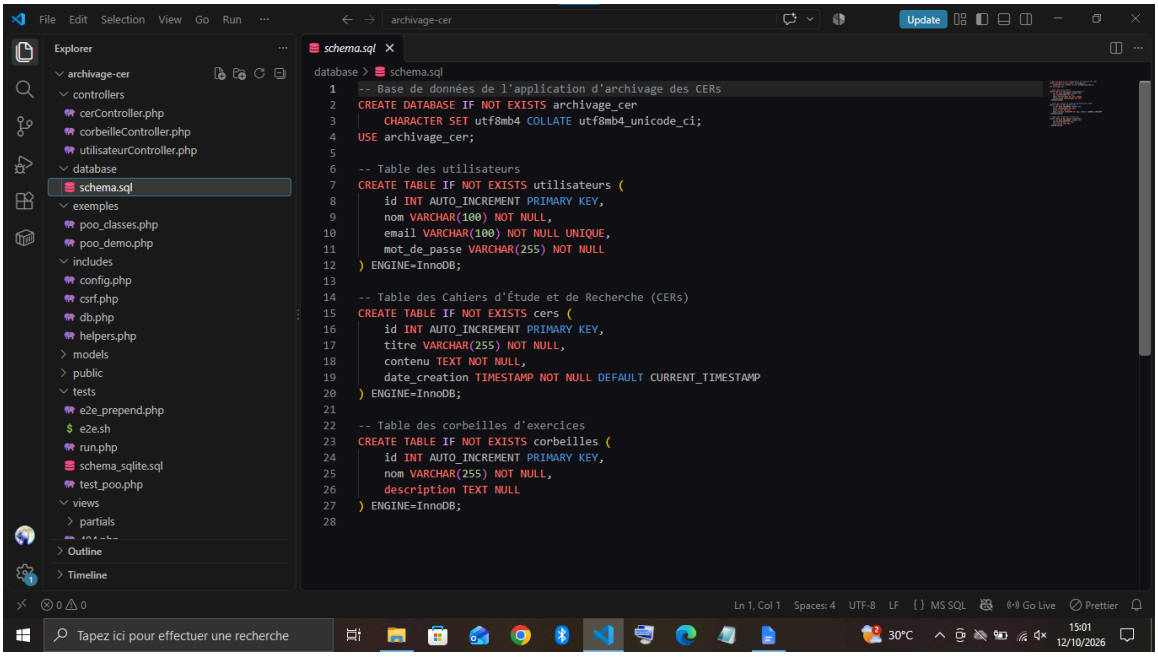
Au terme de ce Prosit, j'ai acquis plusieurs compétences importantes :

1. Définition et utilisation des classes et des objets en PHP ;
2. Application de l'encapsulation, de l'héritage et du polymorphisme ;
3. Conception des tables relationnelles et leurs clés ;
4. Etablissement d'une connexion PDO ;
5. Réalisation des opérations CRUD avec des requêtes préparées ;
6. Organisation d'une API en modèles, dépôts et contrôleurs ;
7. Tests des échanges entre PHP, MySQL et un client HTTP.

Résultats obtenus

Application en php orienté objet avec sécurité des failles. Le code de l'application est disponible via un lien à la fin du document, dans ***liens externes***.



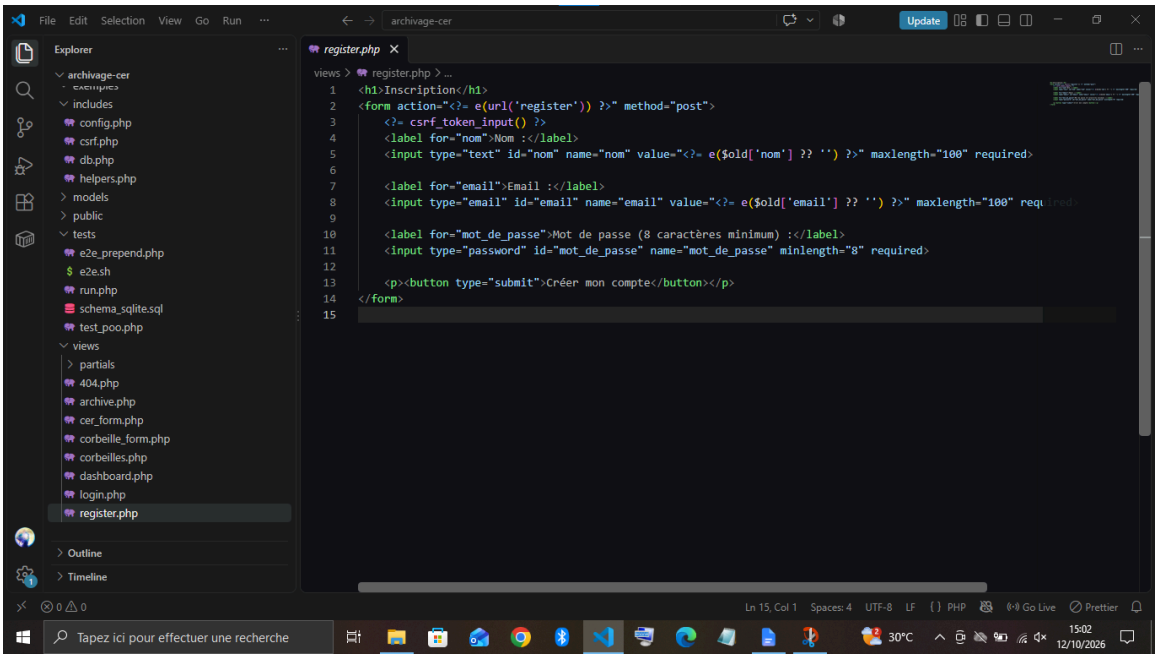


```sql
-- Base de données de l'application d'archivage des CERs
CREATE DATABASE IF NOT EXISTS archivage_cer
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE archivage_cer;

-- Table des utilisateurs
CREATE TABLE IF NOT EXISTS utilisateurs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL
) ENGINE=InnoDB;

-- Table des Cahiers d'Étude et de Recherche (CERs)
CREATE TABLE IF NOT EXISTS cers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,
    contenu TEXT NOT NULL,
    date_creation TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Table des corbeilles d'exercices
CREATE TABLE IF NOT EXISTS corbeilles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    description TEXT NULL
) ENGINE=InnoDB;
```

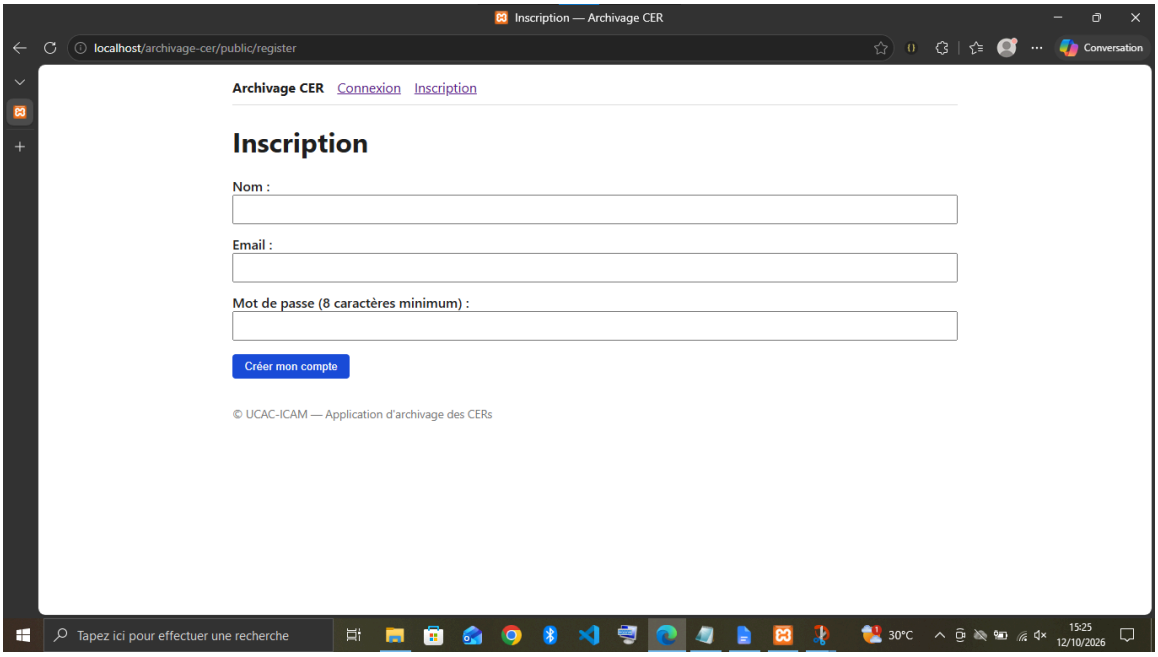


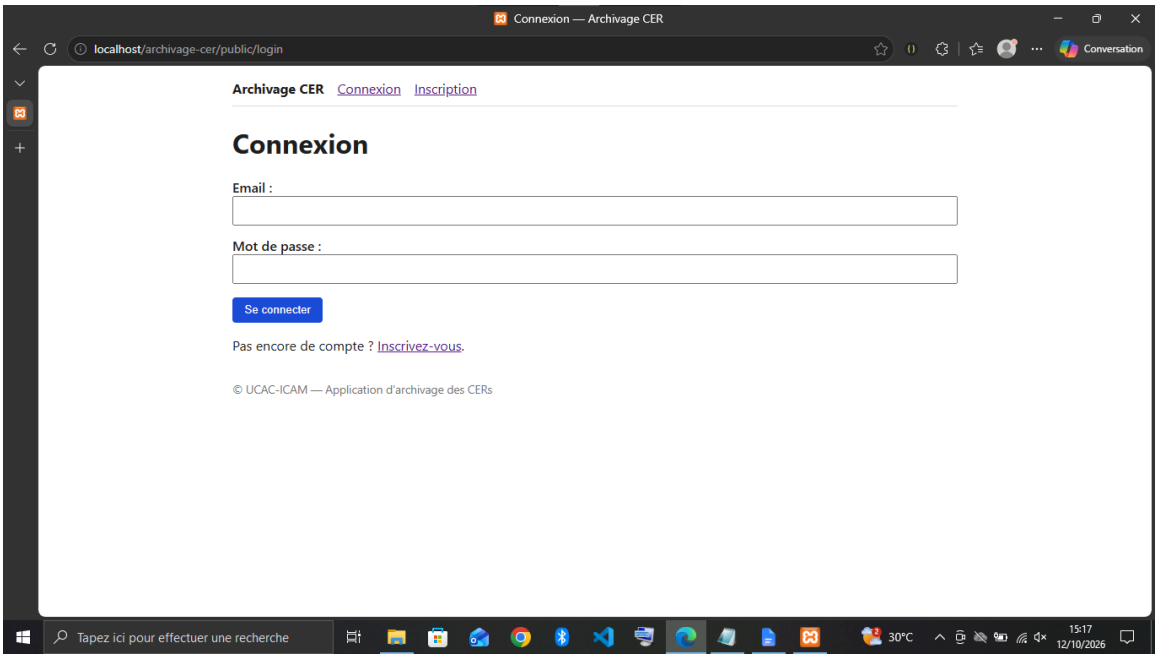
```php
<h1>Inscription</h1>
<form action="<?= e(url('register')) ?>" method="post">
    <?= csrf_token_input() ?>
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom" value="<?= e($old['nom'] ?? '') ?>" maxlength="100" required>

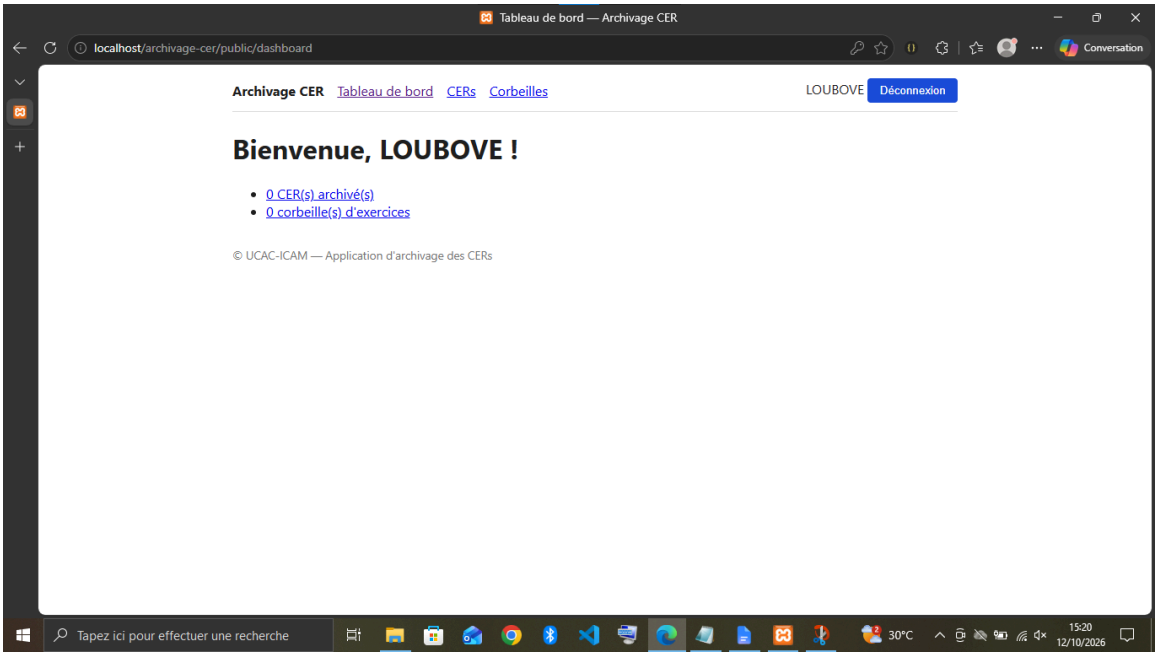
    <label for="email">Email :</label>
    <input type="email" id="email" name="email" value="<?= e($old['email'] ?? '') ?>" maxlength="100" required>

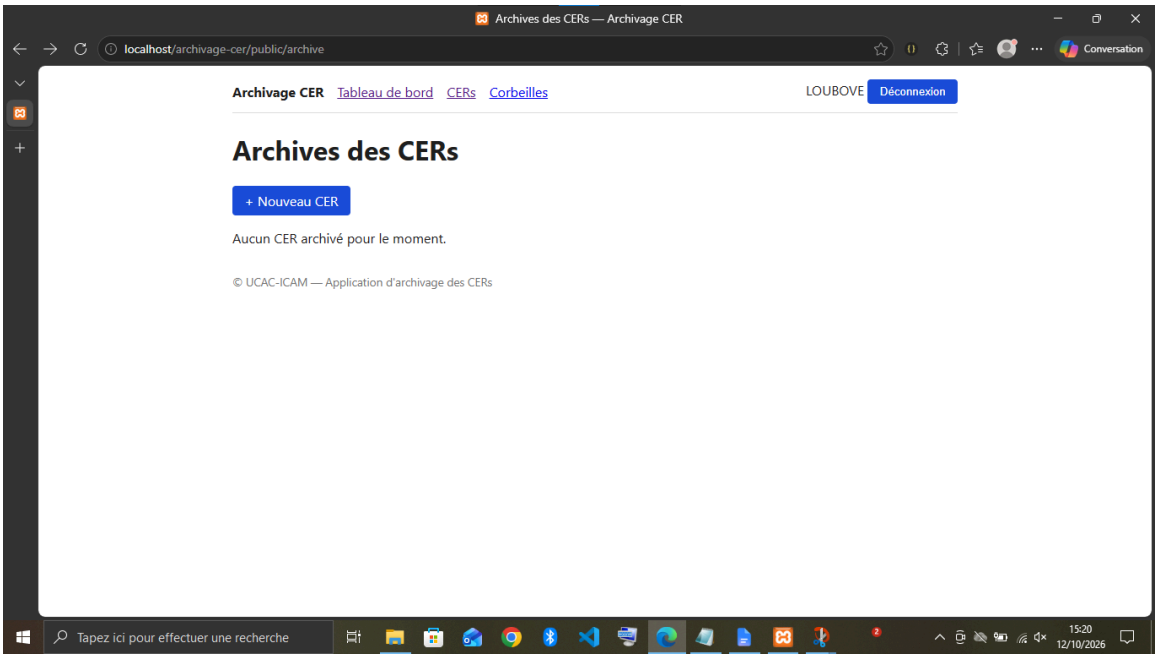
    <label for="mot_de_passe">Mot de passe (8 caractères minimum) :</label>
    <input type="password" id="mot_de_passe" name="mot_de_passe" minlength="8" required>

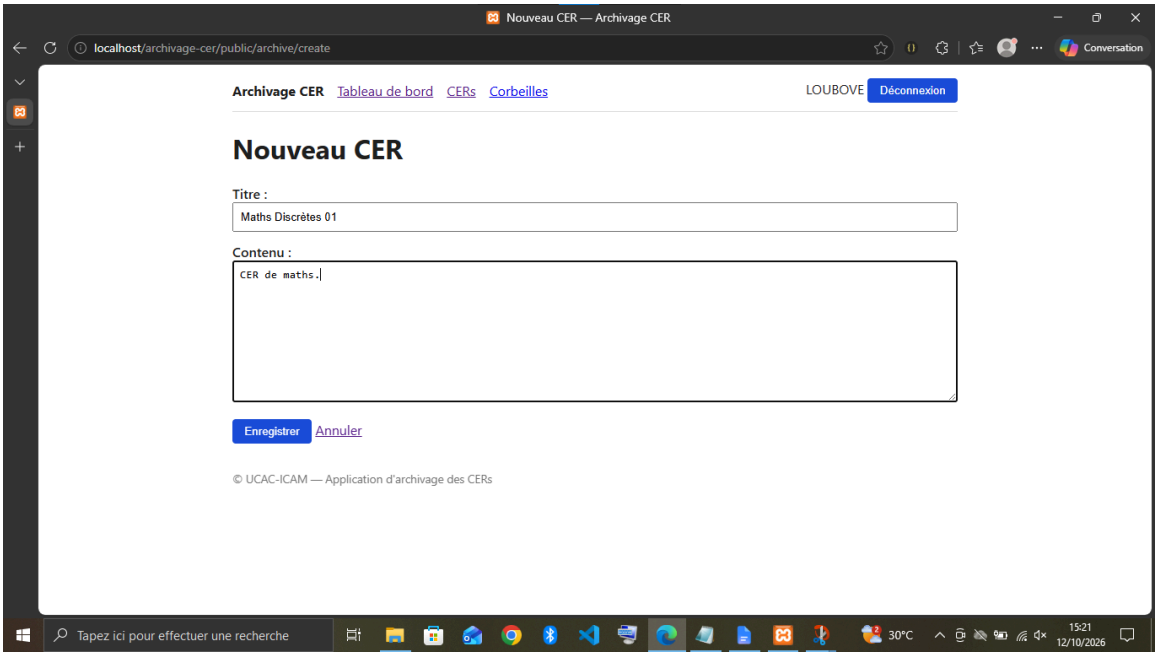
    <p><button type="submit">Créer mon compte</button></p>
</form>
```

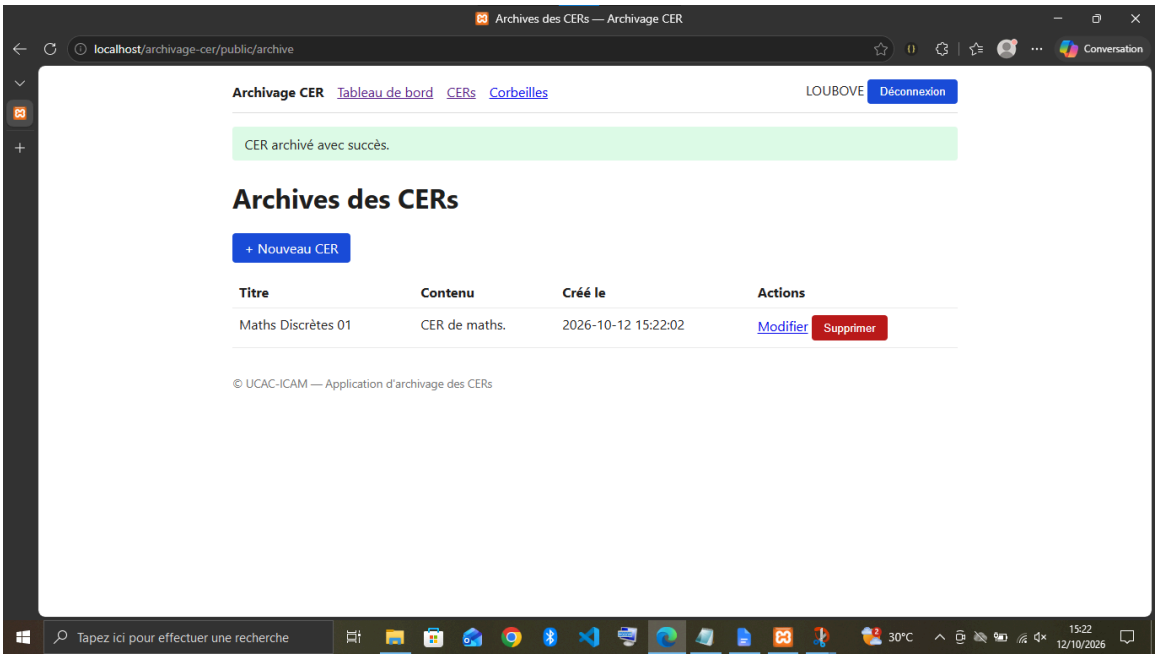

Inscription — Archivage CER
localhost/archivage-cer/public/register
Conversation
Archivage CER Connexion Inscription
Inscription
Nom :
Email :
Mot de passe (8 caractères minimum) :
Créer mon compte
© UCAC-ICAM — Application d'archivage des CERs
Tapez ici pour effectuer une recherche
30°C
15:25
12/10/2026

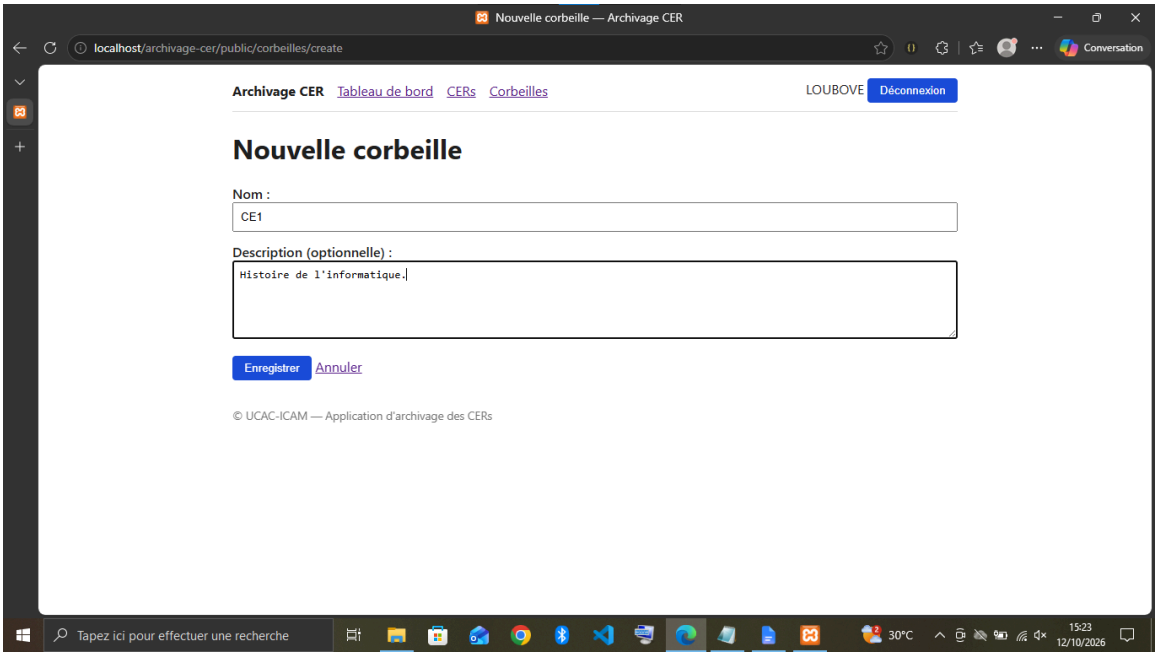

Connexion — Archivage CER
localhost/archivage-cer/public/login
Conversation
Archivage CER Connexion Inscription
Connexion
Email :
Mot de passe :
Se connecter
Pas encore de compte ? Inscrivez-vous.
© UCAC-ICAM — Application d'archivage des CERs
Tapez ici pour effectuer une recherche
30°C
15:17
12/10/2026

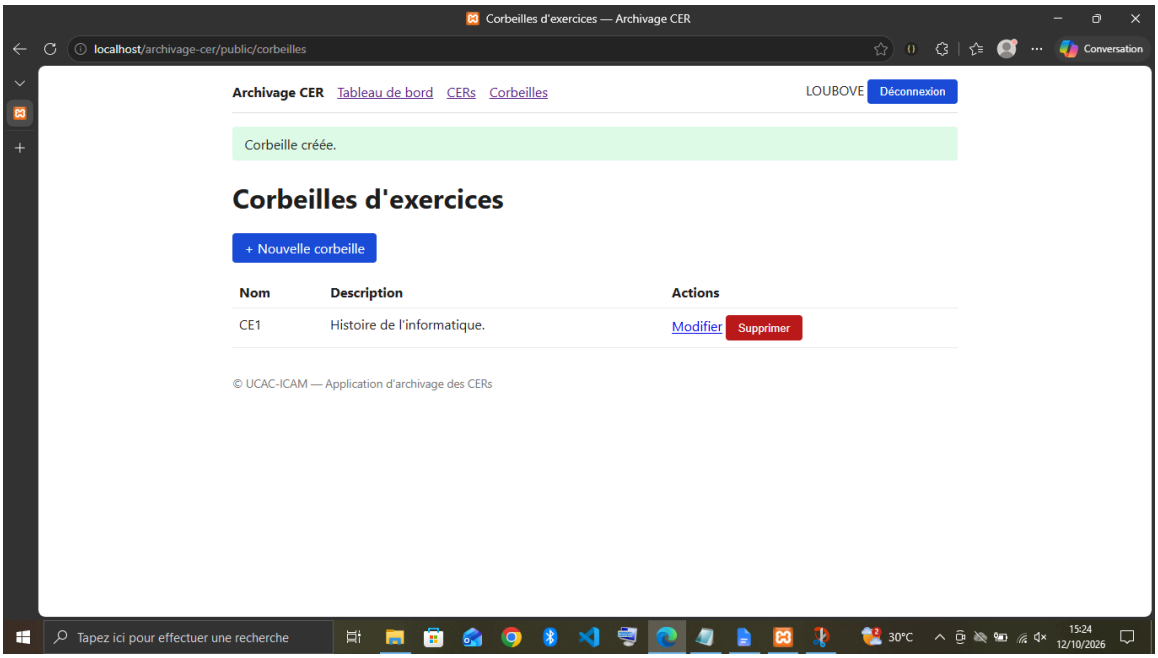

Tableau de bord — Archivage CER
localhost/archivage-cer/public/dashboard
Conversation
Archivage CER
Tableau de bord
CERs
Corbeilles
LOUBOVE
Déconnexion
Bienvenue, LOUBOVE !
0 CER(s) archivé(s)
0 corbeille(s) d'exercices
© UCAC-ICAM — Application d'archivage des CERs
Tapez ici pour effectuer une recherche
30°C
15:20
12/10/2026

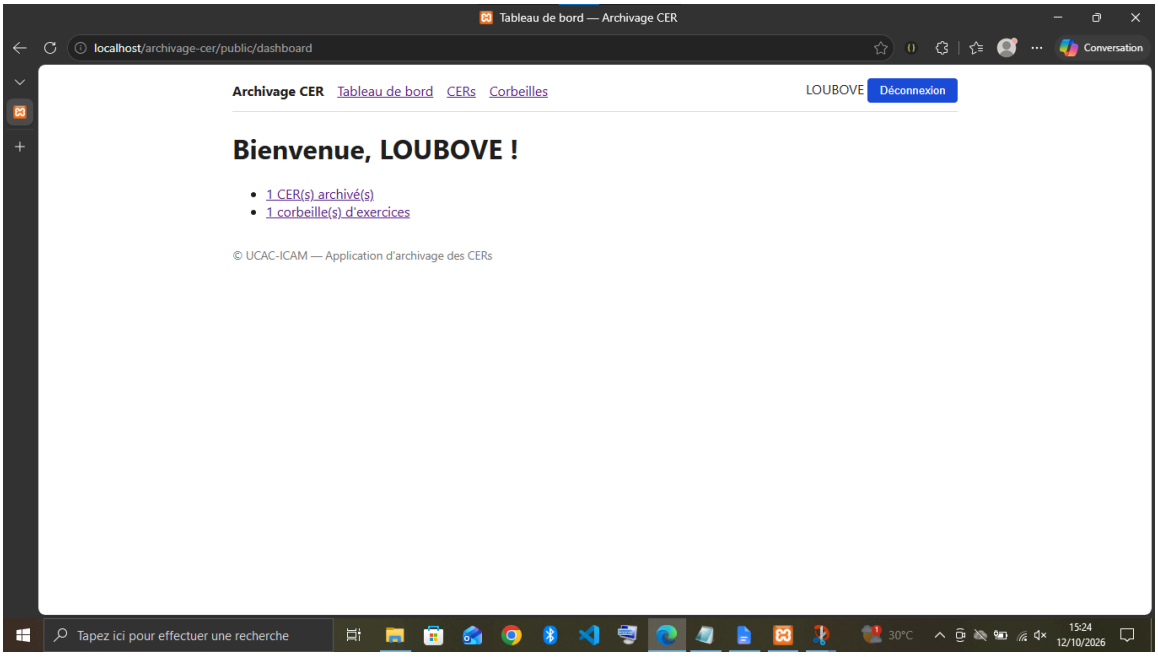

Archives des CERs — Archivage CER
localhost/archivage-cer/public/archive
Conversation
Archivage CER
Tableau de bord
CERs
Corbeilles
LOUBOVE
Déconnexion
Archives des CERs
+ Nouveau CER
Aucun CER archivé pour le moment.
© UCAC-ICAM — Application d'archivage des CERs
Tapez ici pour effectuer une recherche
15:20
12/10/2026


Nouveau CER — Archivage CER
localhost/archivage-cer/public/archive/create
Archivage CER Tableau de bord CERs Corbeilles
LOUBOVE Déconnexion
Nouveau CER
Titre :
Maths Discrètes 01
Contenu :
CER de maths.
Enregistrer Annuler
© UCAC-ICAM — Application d'archivage des CERs


Archives des CERs — Archivage CER
localhost/archivage-cer/public/archive
Archivage CER Tableau de bord CERs Corbeilles
LOUBOVE Déconnexion
CER archivé avec succès.
Archives des CERs
+ Nouveau CER
Titre
Contenu
Créé le
Actions
Maths Discrètes 01
CER de maths.
2026-10-12 15:22:02
Modifier
Supprimer
© UCAC-ICAM — Application d'archivage des CERs


Nouvelle corbeille — Archivage CER
localhost/archivage-cer/public/corbeilles/create
Archivage CER Tableau de bord CERs Corbeilles
LOUBOVE Déconnexion
Nouvelle corbeille
Nom :
CE1
Description (optionnelle) :
Histoire de l'informatique.
Enregistrer Annuler
© UCAC-ICAM — Application d'archivage des CERs


Corbeilles d'exercices — Archivage CER
localhost/archivage-cer/public/corbeilles
Archivage CER Tableau de bord CERs Corbeilles
LOUBOVE Déconnexion
Corbeille créée.
Corbeilles d'exercices
+ Nouvelle corbeille
Nom
Description
Actions
CE1
Histoire de l'informatique.
Modifier Supprimer
© UCAC-ICAM — Application d'archivage des CERs


Tableau de bord — Archivage CER
localhost/archivage-cer/public/dashboard
Conversation
Archivage CER
Tableau de bord
CERs
Corbeilles
LOUBOVE
Déconnexion
Bienvenue, LOUBOVE !
1 CER(s) archivé(s)
1 corbeille(s) d'exercices
© UCAC-ICAM — Application d'archivage des CERs
Tapez ici pour effectuer une recherche
30°C
15:24
12/10/2026

Perspectives

Utiliser un framework PHP comme Laravel afin de bénéficier de performances optimales en terme de sécurité et d’architecture MVC.

8. Références bibliographiques

Références fournies par le prosit

1. **Documentation PHP sur la Programmation Orientée Objet (POO) :**

- PHP.net - Programmation Orientée Objet (POO)
- PHP.net - Classes et Objets

2. **PDO (PHP Data Objects) pour la connexion à MySQL :**

- PHP.net - Introduction à PDO
- PHP.net - Utilisation des requêtes préparées avec PDO

3. **Tutoriels PHP sur les concepts de POO :**

- W3Schools - PHP OOP Tutorial
- TutorialsPoint - PHP OOP Tutorial

4. **Tutoriels sur l'intégration PHP/MySQL avec PDO :**

- DigitalOcean - Guide pour connecter PHP à MySQL avec PDO
- W3Schools - PHP MySQL Database Tutorial

5. **Guides de sécurité des applications Web :**

- OWASP - Guide de prévention des injections SQL
- OWASP - Prévention des attaques CSRF
- OWASP - Guide de prévention des attaques XSS

Ressources Pratiques :

1. **Exemples de code et projets similaires :**

- GitHub - Exemples de projets PHP/MySQL utilisant la POO
- CodeCanyon - Exemples d'applications PHP sécurisées

2. **Validation et assainissement des données en PHP :**

- PHP.net - Validation de données
- W3Schools - PHP Security

3. **Génération et validation de tokens CSRF :**

- OWASP - Token CSRF
- CSRF Token Tutorial en PHP

Outils utilisés

- XAMPP
- VS Code
- Edge
- Powershell
- CMD

Liens externes

Lien github du projet : https://github.com/dloubove/archivage-cer.git